Trabajo tutorizado

Social intelligence y agentes de voz

Big Data Analytics

Profesor D. Juan Manuel Muñoz Domínguez
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «Ejercicios Social Intelligence.pdf» (Campus Virtual UPO).

sería interesante obtener todos los comentarios de una publicación de Facebook para analizarlos o usarlos en tus estrategias de ventas, investigación o marketing? El ejercicio propuesto te enseñará a hacerlo sin necesidad de programar, utilizando herramientas automáticas como Google Sheets, Apify y Make.com.

automatización visual que permite conectar tareas en la web, especialmente diseñada aplicaciones y servicios sin necesidad de para extraer datos (web scraping) y programar. Funciona creando flujos de ejecutar bots sin código o con código trabajo (llamados escenarios) que ejecutan personalizado si lo necesitas. Puedes usar tareas automáticamente cuando se cumplen "actores", que son scripts ya preparados ciertas condiciones. Similares: Zapier, N8N, para tareas comunes como obtener Microsoft Power Automate. comentarios de Facebook, reseñas de Amazon o listados de tiendas.

post google sheet

2. Usar Apify para Scraping de Comentarios

3. Automatizar el Proceso con Make.com

4. Probar y compartir el excel del resultado con el profesor.

Paso 2: Crea una nueva carpeta llamada “Comentarios Facebook".

carpeta, crea una nueva hoja de cálculo de Google Sheets llamada "FComentarios".

posts de los que queremos hacer scrapping de los posts.

Paso 5: Añade una nueva hoja llamada `Datos` y crea las siguientes columnas:

A: `post URL`

B: `comment date`

C: `commenter URL`

D: `commenter name`

E: `commenter text`

F: `likes`

Paso 1: Crea una cuenta en apify.com (incluye crédito gratuito para pruebas). Os pedira el tel para verificar el registro.

• Tened en cuenta que solo ofrece 5 dolares mensuales gratis, pero son mas que suficientes para realizar la practica.

Paso 3: Busca **Facebook Post Comment Scraper** (de easyAPI) y haz clic.

Tened en cuenta que este actor tiene un solo dia de prueba gratis, por tanto os aconsejo leer y enteder la practica antes de realizarla. Para activarlo le damo a start.

- En Facebook, abre una publicación y haz clic en la hora o fecha para abrirla sola.

- Copia la URL desde la barra del navegador

• Paso 4: En la sección de entrada (input), pega la URL de una publicación de Facebook:

• En Facebook, abre una publicación y haz clic en la hora o fecha para abrirla sola.

• Copia la URL desde la barra del navegador

Paso 5: Pega esa URL en Apify, selecciona la cantidad de comentarios a extraer (por ejemplo 20), y haz clic en **Save and Start**.

Paso 6: Una vez finalizado, ve a la pestaña **Output** para ver los comentarios extraídos.

3. Automatizar el Proceso con Make

[make.com](https://ww w.make.com) y entra en tu panel.

Comentarios de Facebook. Lo crearemos desde Scratch.

- Selecciona **Google Sheets**.

- Acción: **Search Rows**.

- Busca **Apify** y selecciona **Run Actor**.

- Conecta tu cuenta de Apify pegando el API Token.

llevara a la pagina de Apify que nos proporcionará la api key.

En "Run synchronously" selecciona **Yes**. Eso hace que el Sistema espere a que termine el scrapping para seguir.

- Está en input en la pestaña JSON de Apify en Make.

• Si no cambiasemos nada el flujo simpre tomaria los comentarios del post que hemos puesto.

• Para hacerlo dinamico tendremos que cambiar el link por una variable.

• Al pinchar ese campo, podremos ver las possibles variables.

Automatizar el Proceso con Make (10)

Get Dataset Items para obtener los resultados:

- Límitalo a 20 resultados

- (puedes ampliarlo luego).

- Selecciona el dataset generado por el actor.

final:

- Selecciona **Google Sheets** y la acción **Add Row**.

de Apify:

- `post URL` → link de la publicación

- `comment date` → timestamp

- `commenter URL` → perfil del comentarista

- `commenter name` → nombre

- `commenter text` → texto del comentario

- `likes` → reacciones

Comprueba el resultado en la hoja de calculo.

de Facebook sin programar ni copiar manualmente. Puedes usarlo para estudios de mercado, detectar oportunidades, analizar audiencias y mucho más.

• Documenta lo realizado y comparte el resultado (hoja de calculo), para que sea evaluado.

• Comparte el flujo (Exportalo de make)

• Comparte el link del post que has utilizado presente vuestro servicio a un cliente.

– Agente que llamara con su script (Eleven Labs)

– Herramientas que se conecten a make en Eleven labs

– Flujos en make, aquí se expone como crear el flujo básico es parte de la practica el crear el flujo completo.

Creación del numero en twilo agente de voz

Vamos a comprar número y compramos un numero con el crédito gratuito, este numero tiene bastantes restricciones pero nos servirá para hacer las pruebas que haremos a continuación. Realmente lo queremos para hacer llamadas así que lo principal es que no sea local ya que los mas baratos son los de US.

número para España números verificados. La cuenta Trail tiene un número reducido de números verficados.

secciones, nosotros nos centraremos en la sección de Plataforma de agentes.

Creación de un Agente

Prompt del sistema critica y creativa del sistema. Se llevará la mayor parte del tiempo.

Selección de la voz

Idioma y LLM temperatura del sistema así como el número de tokens.

información adicional hacer una prueba, sin realizar ninguna llamada, la principal ventaja es que el coste de esta llamada es del 50% de créditos de Eleven Labs.

compramos en twilo

Recordad lo que generamos en twilo la llamada, como es la versión trial solo podrás llamar a numero verificados, además nos dira en la llamada en ingles que es la versión trial y que debes pulsar un botón para seguir.

acciones que queramos

```
{{nombre}}
{{telefono}}
{{tipo_cita}}
{{fecha}}
{{hora}}
```

puede cambiarse a JSON

Cuando creamos el webhook nos dará la dirección de escucha, que utilizaremos en Elevenlabs

Hoja de calculo

Prompt utilizado en Eleven labs.

Json de las variables utilizadas para las herramientas creadas.

Flujos exportado de Make

Grabación de como se comporta vuestro Agente.

Ejercicio 3

Numero creado en Prompt utilizado en utilizadas para las Twilo (Pantallazo) Eleven labs. herramientas creadas.

Grabación de como Flujos exportado de Screenshot de se comporta Make vuestro calendario vuestro Agente.

RESOLUCIÓN

Contexto: el negocio sobre el que se trabaja

Los tres ejercicios se resuelven sobre una misma idea de negocio, **EcoAsesores**, una consultora energética que ayuda a pymes y autónomos a reducir su factura de luz y gas mediante un análisis gratuito de la factura. Es un negocio que encaja bien con los tres ejercicios porque combina las dos caras del *social intelligence*: **escuchar** lo que dice el mercado (ejercicio 1) y **hablar** con el cliente de forma automatizada (ejercicios 2 y 3).

Ejercicio 1. Evaluación de un post de Facebook con Apify

El objetivo es extraer automáticamente todos los comentarios de una publicación de Facebook y depositarlos en una hoja de cálculo, sin programar, encadenando tres herramientas: **Google Sheets** (entrada y salida), **Apify** (extracción) y **Make** (orquestación).

Por qué tiene valor extraer los comentarios de un post

Los comentarios son la fuente de opinión más barata y menos filtrada que existe. A diferencia de una encuesta, nadie los escribe sabiendo que van a ser analizados, y a diferencia de las métricas de alcance, contienen el *porqué* y no sólo el *cuánto*. Para EcoAsesores, leer sistemáticamente los comentarios de publicaciones sobre facturas de luz permite tres cosas concretas: detectar el vocabulario real con el que la gente describe su problema (que es el que hay que usar en los anuncios), identificar objeciones recurrentes antes de que las plantee un comercial, y localizar clientes potenciales que se están quejando en público de su compañía actual.

Paso 1. Preparación de Google Drive y Google Sheets

En Google Drive, con la cuenta `tu-cuenta@gmail.com`, se crea la carpeta **Comentarios Facebook** y dentro de ella la hoja de cálculo **FComentarios**. La hoja se estructura en dos pestañas con papeles bien diferenciados, que es lo que permite que el flujo sea reutilizable: una pestaña de **entrada**, donde se van apuntando las publicaciones que se quieren vigilar, y una pestaña de **salida**, donde el flujo deposita los comentarios extraídos.

Pestaña	Papel en el flujo	Contenido
`URLs`	Entrada. La lee el primer módulo de Make.	Columna A: `post URL`. Una fila por publicación a analizar.
`Datos`	Salida. La escribe el último módulo de Make.	A: `post URL` · B: `comment date` · C: `commenter URL` · D: `commenter name` · E: `commenter text` · F: `likes`

La cabecera de `Datos` se escribe a mano una única vez porque el módulo *Add a Row* de Make mapea los campos por posición de columna: si la cabecera no existe, Make no ofrece los nombres de columna al configurar el mapeo.

Paso 2. Extracción de los comentarios con Apify

Apify es un mercado de *actores*: programas de scraping ya escritos que se ejecutan en su infraestructura y dejan el resultado en un *dataset*. El guion propone el actor *Facebook Post*

Comment Scraper de easyAPI, pero ese actor sólo ofrece **un día de prueba gratuita** y en el momento de realizar la práctica ya no era utilizable con la cuenta. Se ha usado en su lugar **Facebook Comment Scraper** (`dz_omar/facebook-comment-scraper`, id `K5EXlxalV2BCYfgKM`), que hace exactamente lo mismo —recibe una o varias URL de publicaciones y devuelve los comentarios— y se factura por evento dentro de los 5 $ mensuales gratuitos. El cambio de actor no altera el ejercicio: lo que se evalúa es la cadena Sheets → Apify → Make.

La publicación elegida es una de **Humans of New York**, una página con volumen alto de comentarios reales y en abierto, lo que da material suficiente para una prueba realista:

`https://www.facebook.com/humansofnewyork/posts/pfbid02CTjAT521p7UqCMY9c5B2oJKvDB18Sdzuk1brceTFvhC6d3mhEccj9TeRXVpn7Nv4l`

La ejecución manual del actor desde la consola de Apify sirve para dos cosas: comprobar que la URL es válida y, sobre todo, **obtener el JSON de entrada** que después hay que pegar en Make. Ese JSON es:

```
{
  "fetchReplies": false,
  "maxCommentsPerUrl": 20,
  "urls": [
    { "url": "https://www.facebook.com/humansofnewyork/posts/pfbid02CTjAT..." }
  ],
  "commentsIntentToken": "RANKED_FILTERED_INTENT_V1",
  "maxRepliesPerComment": 0
}
```

El resultado de la ejecución fue **correcto**: 20 comentarios extraídos en 21 segundos con un coste de **0,032 $**. Cada elemento del dataset es un objeto anidado con la forma `comment.author.name`, `comment.author.url`, `comment.text`, `comment.created_at`, `comment.total_reactions`, etc.; esos son los nombres que habrá que usar al mapear las columnas en Make.

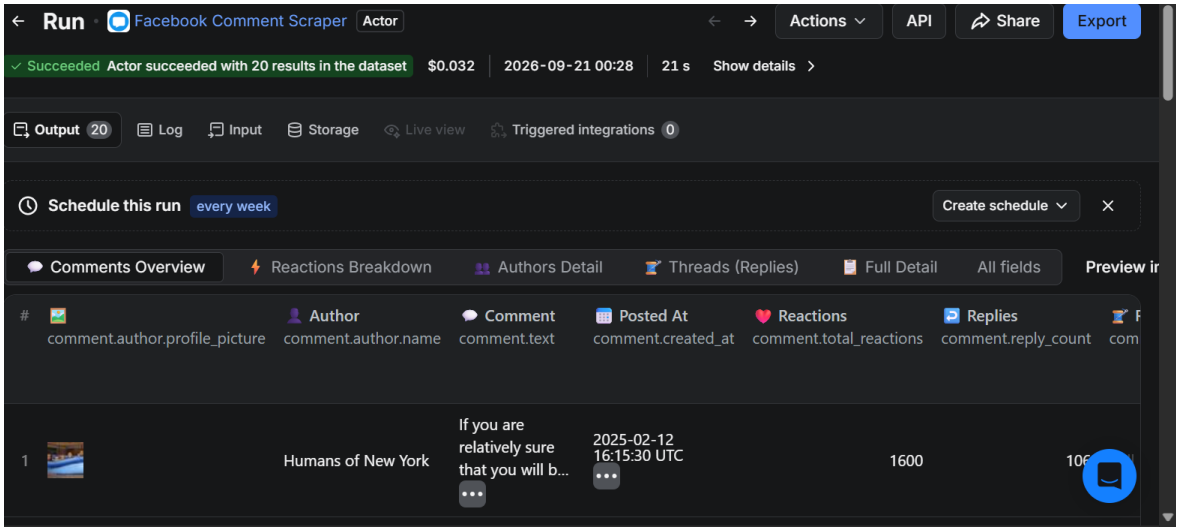


Figura 1. Consola de Apify: la ejecución del actor *Facebook Comment Scraper* termina con estado **Succeeded** y 20 comentarios en el dataset.

Paso 3. Automatización del proceso en Make

El escenario se llama **Comentarios de Facebook** y encadena cuatro módulos. La idea de fondo es que ningún dato esté escrito a mano dentro del flujo: la URL entra desde la hoja, el identificador del dataset sale de la propia ejecución del actor y las columnas se rellenan con los campos que devuelve Apify. Así, para analizar otra publicación basta con añadir una fila en la pestaña `URLs`.

#	Módulo	Configuración relevante
1	Google Sheets · *Search Rows*	Conexión OAuth con `tu-cuenta@gmail.com`. Hoja `FComentarios`, pestaña `URLs`, *Table contains headers* = Yes, rango `A-CZ`. Devuelve un paquete por publicación a vigilar.
2	Apify · *Run an Actor*	Actor `dz_omar/facebook-comment-scraper`. **Run synchronously = Yes**, para que el flujo espere a que termine el scraping. En el JSON de entrada la URL fija se sustituye por la variable del módulo 1 que corresponde a la columna A.
3	Apify · *Get Dataset Items*	*Dataset ID* = `{{2.defaultDatasetId}}`, es decir, el dataset que acaba de generar el módulo 2. *Limit* = 20, *Format* = JSON, *Data transformation* = Clean. Devuelve 20 paquetes, uno por comentario.
4	Google Sheets · *Add a Row*	Misma conexión, hoja `FComentarios`, pestaña `Datos`. Mapeo de las seis columnas (ver más abajo). Se ejecuta una vez por comentario, de ahí los 20 registros.

El mapeo de campos

Es la parte del ejercicio en la que hay que entender la estructura del JSON de Apify, porque los comentarios llegan anidados dentro de un objeto `comment`. Make identifica las columnas del módulo de lectura por su posición, de modo que la columna A del módulo 1 se escribe con la sintaxis que aparece en el blueprint de la página siguiente:

Columna de la hoja	Origen
`post URL`	Columna A del módulo 1 — la URL leída de la pestaña `URLs`
`comment date`	`{{3.comment.created_at}}`
`commenter URL`	`{{3.comment.author.url}}`
`commenter name`	`{{3.comment.author.name}}`
`commenter text`	`{{3.comment.text}}`
`likes`	`{{3.comment.total_reactions}}`

Dos errores que conviene documentar. (1) El primer intento falló con *«Missing value of required parameter datasetId»*: se había mapeado el campo `defaultDatasetId` desde el grupo *Data* que muestra el panel de variables, pero en la salida real ese campo cuelga de la raíz del paquete, de modo que la expresión correcta es `{{2.defaultDatasetId}}`. (2) Make ofrece los nombres de columna del módulo

4 leyendo la fila 1 de la pestaña de destino; si esa fila no está bien formada, las columnas aparecen como `(A)`, `(B)`, `(C)`… y el mapeo se vuelve ilegible. Basta con corregir la cabecera en la hoja y pulsar *Refresh* en el módulo.

Paso 4. Ejecución y resultado

Con el escenario guardado se lanza con **Run once**. Los cuatro módulos terminan en verde y el contador del último módulo marca **20 operaciones**: una fila escrita por cada comentario extraído.

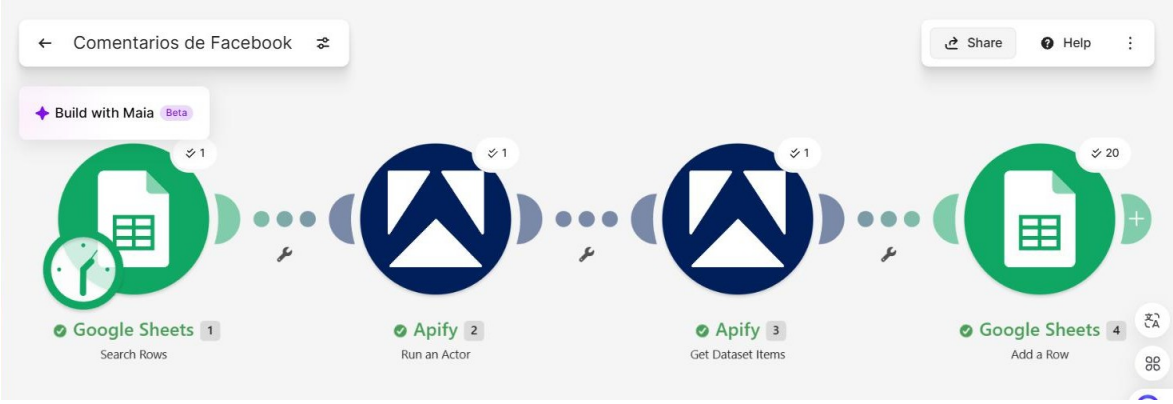


Figura 2. Escenario **Comentarios de Facebook** ejecutado con *Run once*: 1 fila leída de `URLs`, 1 ejecución del actor, 1 dataset y **20 filas** escritas en `Datos`.

En la hoja de cálculo aparecen los 20 comentarios con sus seis campos. Se observa que `commenter URL` sólo viene informado para los perfiles que Facebook expone públicamente (páginas y perfiles abiertos); en los demás llega vacío, lo cual es el comportamiento normal del actor y no un fallo del flujo.

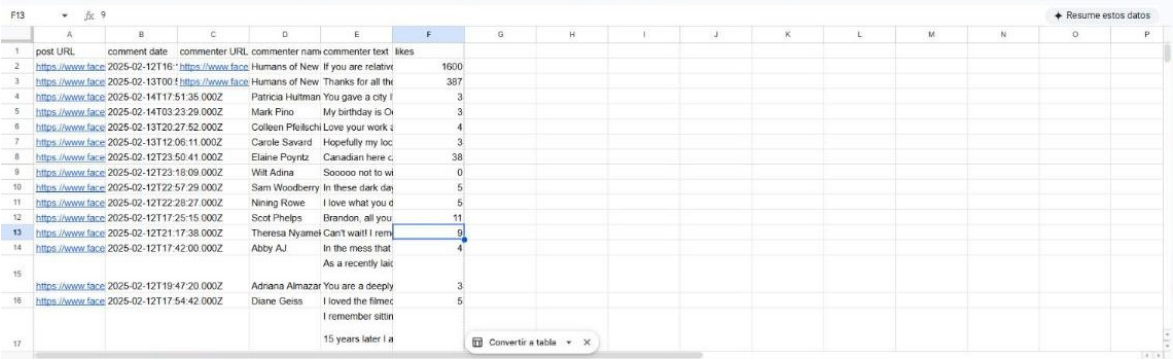


Figura 3. Pestaña `Datos` de **FComentarios** tras la ejecución: 20 comentarios con fecha, autor, texto y número de reacciones.

Blueprint exportado del escenario

Make permite exportar el escenario completo como JSON (*Export blueprint*). Se entrega el fichero `Comentarios de Facebook.blueprint.json` junto con este documento. Su núcleo, con los cuatro módulos y sus mapeos, es el siguiente:

```
"flow": [
  { "id": 1, "module": "google-sheets:filterRows", "version": 2,
    "mapper": { "from": "drive", "spreadsheetId": "1jQ-V4CE_...KKd1aE",
```

```
                "sheetId": "URLs", "includesHeaders": true,
                "tableFirstRow": "A1:CZ1", "sortOrder": "asc" } },

  { "id": 2, "module": "apify:runActorNew", "version": 1,
    "mapper": { "actor_search_type": "user", "runSync": true,
                "actorId": "K5EXlxalV2BCYfgKM",
                "inputBodyK5EXlxalV2BCYfgKM":
                  "{\"fetchReplies\": false, \"maxCommentsPerUrl\": 20,
                    \"urls\": [{\"url\": \"{{1.`0`}}\"}],
                    \"commentsIntentToken\": \"RANKED_FILTERED_INTENT_V1\",
                    \"maxRepliesPerComment\": 0}" } },

  { "id": 3, "module": "apify:fetchDatasetItems", "version": 1,
    "mapper": { "datasetId": "{{2.defaultDatasetId}}",
                "type": "clean", "format": "json", "limit": "20" } },

  { "id": 4, "module": "google-sheets:addRow", "version": 2,
    "mapper": { "spreadsheetId": "1jQ-V4CE_...KKd1aE", "sheetId": "Datos",
                "includesHeaders": true, "insertDataOption": "INSERT_ROWS",
                "values": { "0": "{{1.`0`}}",
                            "1": "{{3.comment.created_at}}",
                            "2": "{{3.comment.author.url}}",
                            "3": "{{3.comment.author.name}}",
                            "4": "{{3.comment.text}}",
                            "5": "{{3.comment.total_reactions}}" } } }
]
```

Qué se entrega y para qué sirve

- Hoja de cálculo **FComentarios** (carpeta *Comentarios Facebook* de Google Drive), con la pestaña `URLs` de entrada y la pestaña `Datos` con los 20 comentarios extraídos.
- Blueprint del escenario de Make: `Comentarios de Facebook.blueprint.json`.
- Publicación analizada: `https://www.facebook.com/humansofnewyork/posts/pfbid02CTjAT521p7UqCMY9c5B2oJKvDB18Sdzuk1brceTFvhC6d3mhEccj9TeRXVpn7Nv4l`

El flujo queda operativo y reutilizable: añadiendo filas a `URLs` y activando la programación del escenario (Make permite ejecutarlo cada 15 minutos, cada día o por webhook), EcoAsesores tendría una escucha automática de las publicaciones de su sector, con los comentarios acumulándose en una hoja lista para analizar —conteo de menciones, análisis de sentimiento o simplemente lectura ordenada por reacciones.

Ejercicios 2 y 3. Agente de voz que presenta el servicio y agenda la cita

Los ejercicios 2 y 3 son el mismo sistema en dos niveles de exigencia: primero un agente que llama, presenta el servicio de EcoAsesores y concierta una cita; después ese mismo agente consultando la agenda real para comprobar si el hueco está libre y, si no lo está, proponiendo alternativas. Por eso se resuelven sobre una única implementación y se documentan juntos, señalando en cada punto qué parte corresponde a cada ejercicio.

Decisión técnica: VAPI en lugar de Twilio + ElevenLabs + Make

El guion propone montar el sistema con tres piezas: un **número de teléfono de Twilio**, un **agente de ElevenLabs** que habla por ese número y un **webhook de Make** que ejecuta las acciones (guardar en una hoja, enviar un correo). Esta solución se ha construido sobre **VAPI**, una plataforma que integra en un mismo producto el reconocimiento de voz, el modelo de lenguaje, la síntesis de voz y la llamada a herramientas. Los motivos son concretos:

- **No hace falta comprar un número.** La cuenta *trial* de Twilio sólo permite llamar a números previamente verificados y antepone un mensaje en inglés a cada llamada, lo que ensucia la grabación que hay que entregar. VAPI permite lanzar la llamada desde el navegador (*Talk to Assistant*) con el mismo agente, el mismo prompt y las mismas herramientas; si mañana se quiere telefonía real, basta con asociar un número al asistente sin tocar nada más.
- **Las herramientas son nativas.** En el esquema del guion, ElevenLabs no puede leer un calendario: hay que montar un webhook en Make que haga de intermediario. VAPI incorpora herramientas de Google Calendar conectadas por OAuth, de modo que el agente consulta la disponibilidad y crea el evento directamente. Se pierde el paso por Make, pero se gana exactamente lo que pide el ejercicio 3: que el agente compruebe huecos y agende de verdad.
- **La extracción estructurada viene incluida.** Lo que en el guion se resuelve definiendo variables `{{nombre}}`, `{{fecha}}`… y enviándolas a un webhook, aquí se resuelve con una *structured output*: al colgar, VAPI analiza la conversación y devuelve un JSON con los campos declarados. Es el mismo resultado —un objeto con los datos de la llamada listo para enrutar— con menos piezas intermedias.

Se mantiene, por tanto, todo lo que el enunciado evalúa (prompt, herramientas con su JSON de variables, comportamiento del agente, grabación y evidencia en el calendario) y se sustituye la fontanería por una equivalente. La única entrega del guion que no aplica es la captura del número de Twilio, porque no se ha comprado número.

El negocio y el objetivo del agente

EcoAsesores es una consultora energética para pymes y autónomos. Su gancho comercial es un análisis gratuito de la factura de luz o gas, con un ahorro medio declarado de entre el 15 % y el 25 %. La llamada no vende: su único objetivo es conseguir una cita de 30 minutos con un asesor. A esto se le ha añadido un segundo objetivo, pedido expresamente para este trabajo: **captar el motivo real de la llamada del cliente y generar un texto de derivación al departamento que corresponda**.

Arquitectura de la solución

Pieza	Tecnología elegida	Equivalente en el guion
Canal de voz	Llamada web de VAPI (*Talk to Assistant*); el asistente admite número asociado sin cambios	Número comprado en Twilio
Reconocimiento de voz	Soniox STT RT v5, multilingüe (español)	Incluido en ElevenLabs
Cerebro	GPT-4.1 con el *system prompt* de Marta	LLM configurable de ElevenLabs
Voz	Elliot v2 (VAPI)	Voz de ElevenLabs
Herramientas	Google Calendar nativo en VAPI (OAuth con `tu-cuenta@gmail.com`)	Webhook de Make llamado desde ElevenLabs
Salida de datos	*Structured output* `derivacion_departamento` (extracción automática al colgar)	Variables `{{nombre}}`, `{{fecha}}`… enviadas al webhook

El asistente se llama **Marta - EcoAsesores** (identificador `e8e396bf-d50a-4ee0-863f-cd125e3a708e`) y está publicado en su **versión 6**, que es la que se ha usado para la llamada que se entrega.

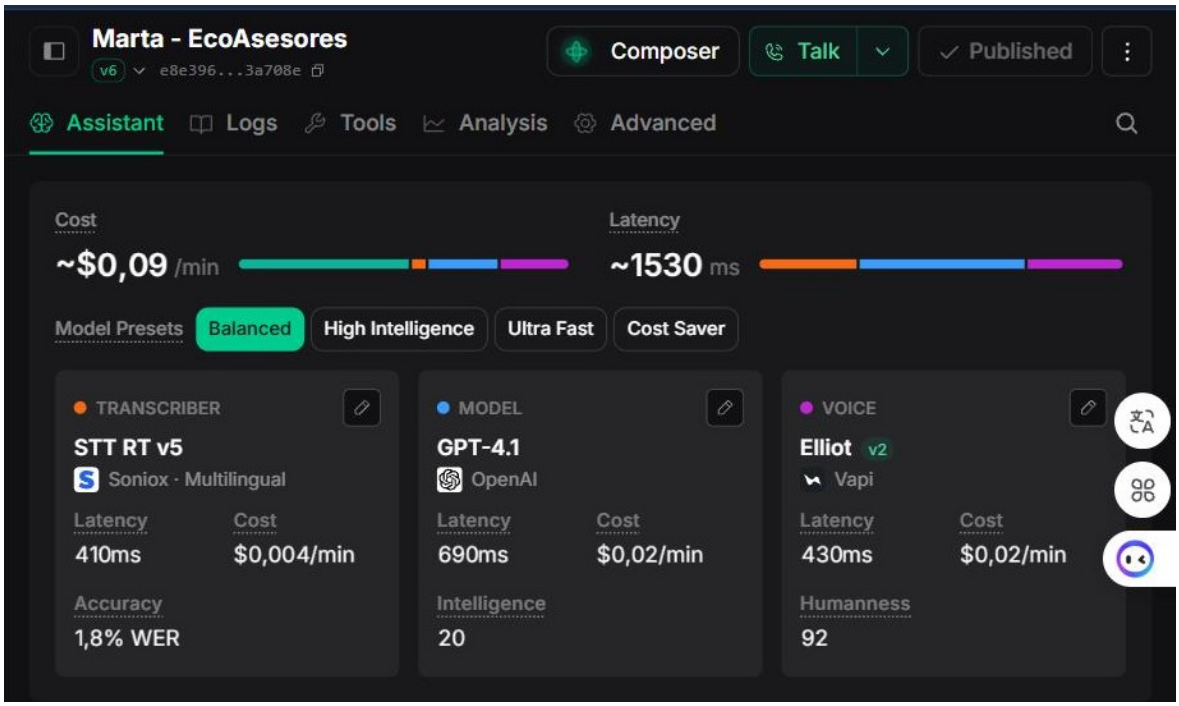


Figura 4. Asistente **Marta - EcoAsesores**, versión 6 publicada: transcripción con Soniox multilingüe, modelo GPT-4.1 y voz Elliot. Coste estimado 0,09 $/min y latencia de 1,5 s.

El prompt: dónde está realmente el trabajo

El guion lo dice con razón: *el prompt es la parte crítica y creativa del sistema*. El de Marta está organizado en bloques con una función cada uno, y cada bloque responde a un problema observado en las pruebas.

Bloque	Para qué está
`OBJETIVO UNICO`	Evita que el agente se disperse: no vende ni da precios, sólo consigue la cita.
`ESTILO`	Español de España, frases cortas, máximo dos

	preguntas seguidas y prohibición de insistir más de una vez. Sin esto el modelo encadena preguntas y suena a interrogatorio.
`GUION` (5 pasos)	Saludo e identificación, cualificación, propuesta de valor, cierre con dos alternativas de hora y confirmación en voz alta.
`HERRAMIENTAS`	Le dice cuándo usar cada herramienta de calendario y qué hacer si el hueco está ocupado. Es el bloque que implementa el **ejercicio 3**.
`CONTEXTO TEMPORAL`	Inyecta la variable dinámica `{{now}}` de VAPI con la zona `Europe/Madrid` y le prohíbe decir ninguna fecha que no haya deducido de ahí o que no le haya devuelto el calendario.
`DATOS DEL CLIENTE`	Le obliga a pedir nombre y apellidos nada más saludar y a confirmarlos en voz alta; se necesitan para el título del evento y para el texto de derivación.
`REGLAS`	No inventar tarifas ni huecos, no pedir DNI ni datos bancarios, tramitar la baja de contacto si la piden y cerrar resumiendo el compromiso.
`CAPTACION DEL MOTIVO Y DERIVACION`	Las seis categorías (ALTA, BAJA, FACTURACION, AVERIA, COMERCIAL, OTROS), el departamento al que va cada una y el resumen que debe redactar y confirmar con el cliente.

Los dos bloques resaltados nacieron de fallos reales. En una prueba temprana el agente **se inventó la fecha**: dijo «miércoles 19 de junio» y la salida estructurada devolvió `2024-06-19`, porque el modelo no tiene noción del día actual y rellenó el hueco con lo primero que le sonaba plausible. La solución es darle la fecha como dato, no como suposición: el bloque `CONTEXTO TEMPORAL` con `{{now}}`. En esa misma prueba tampoco preguntó el nombre del cliente, y sin nombre no hay título de cita ni derivación identificable; de ahí el bloque `DATOS DEL CLIENTE`. Ambos se añadieron y se publicó la versión 6, con la que el comportamiento ya fue correcto.

Las herramientas y su JSON

El asistente tiene asociadas dos herramientas nativas de Google Calendar, ambas conectadas por OAuth contra el calendario `tu-cuenta@gmail.com` con zona horaria `Europe/Madrid`. La primera consulta, la segunda escribe; esa separación es lo que permite el comportamiento del ejercicio 3.

Herramienta	Tipo VAPI	Papel
`google_calendar_check_availability_tool`	`google.calendar.availability.check`	Consulta los huecos libres de la agenda para un rango de fechas. El prompt obliga a usarla **antes** de proponer ninguna hora.
`google_calendar_tool`	`google.calendar.event.create`	Crea el evento de 30 minutos cuando el cliente acepta, con el nombre del cliente en el título y el motivo de la llamada en la descripción.

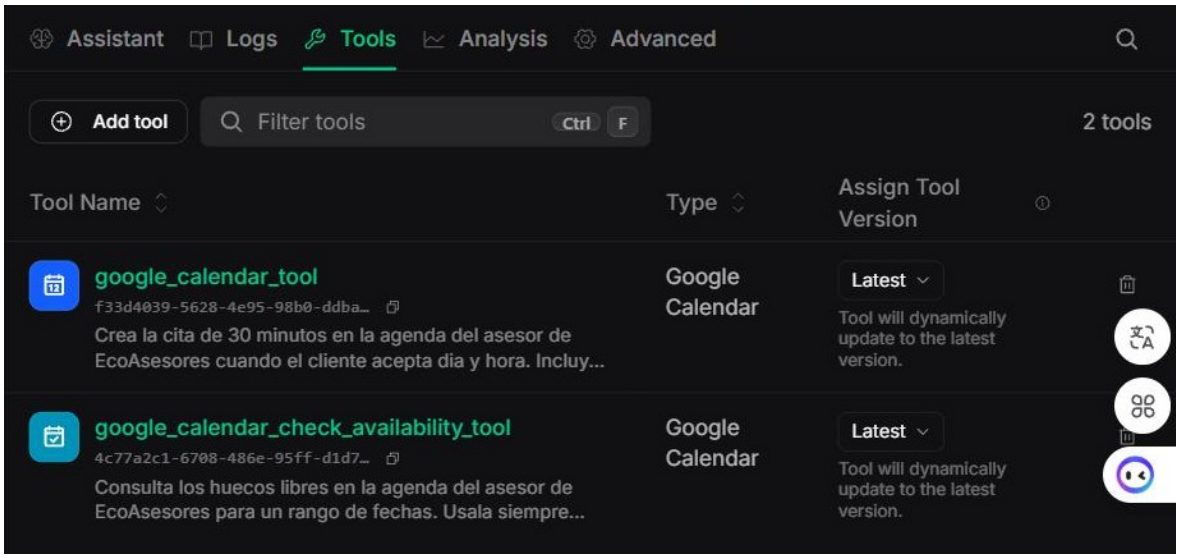


Figura 5. Herramientas asociadas al asistente: creación de cita y consulta de disponibilidad, ambas de tipo Google Calendar.

Este es el JSON completo de la herramienta de creación de cita, tal y como lo devuelve VAPI:

```
{
  "id": "f33d4039-5628-4e95-98b0-ddbae7114e60",
  "type": "google.calendar.event.create",
  "function": {
    "name": "google_calendar_tool",
    "description": "Crea la cita de 30 minutos en la agenda del asesor
       de EcoAsesores cuando el cliente acepta dia y hora. Incluye en el
       titulo el nombre del cliente y en la descripcion el motivo de la
       llamada y los datos recogidos.",
    "parameters": { "type": "object", "properties": {}, "required": [] }
  },
  "messages": [ { "type": "request-start", "blocking": false } ],
  "metadata": {
    "calendarId": "tu-cuenta@gmail.com",
    "timeZone": "Europe/Madrid"
  },
  "latestVersion": "v2"
}
```

Y este el de la herramienta de consulta de disponibilidad, que es la que hace posible el ejercicio 3:

```
{
  "id": "4c77a2c1-6708-486e-95ff-d1d7fba8cc55",
  "type": "google.calendar.availability.check",
  "function": {
    "name": "google_calendar_check_availability_tool",
    "description": "Consulta los huecos libres en la agenda del asesor de
       EcoAsesores para un rango de fechas. Usala siempre antes de proponer
       una hora de cita al cliente, y si la hora que pide esta ocupada, para
       ofrecerle las alternativas libres mas cercanas.",
    "parameters": { "type": "object", "properties": {}, "required": [] }
  },
  "messages": [ { "type": "request-start", "blocking": false } ],
  "metadata": {
```

```
    "calendarId": "tu-cuenta@gmail.com",
    "timeZone": "Europe/Madrid"
  },
  "latestVersion": "v2"
}
```

Conviene explicar por qué `parameters.properties` está vacío. En una herramienta genérica (un webhook) el desarrollador declara a mano las variables `{{nombre}}`, `{{fecha}}`, `{{hora}}`… En las herramientas nativas de VAPI el esquema de argumentos lo aporta la propia integración de Google Calendar (título, inicio, fin, zona horaria, invitados), de modo que lo único que hay que configurar es **a qué calendario** y **en qué zona horaria** escribe, que es lo que viaja en `metadata`. El modelo rellena el resto a partir de la conversación. El equivalente exacto del `{{nombre}}`, `{{fecha}}`, `{{hora}}` del guion es, en esta solución, la salida estructurada que se describe a continuación.

La salida estructurada: captar el motivo y derivar al departamento

El encargo añadido a los ejercicios era que el agente **captase el motivo de la llamada y, una vez captado, generase un texto para derivarlo al departamento oportuno**. Se ha resuelto con una *structured output* de VAPI llamada `derivacion_departamento`: un objeto cuyos campos VAPI extrae automáticamente de la conversación cuando la llamada termina, usando el propio modelo como extractor. No hay que programar nada: se declara el esquema y se asocia al asistente.

Campo	Tipo	Contenido
`cliente`	string	Nombre y apellidos del cliente, tal y como los ha confirmado en voz alta.
`categoria`	string	Una de las seis: ALTA, BAJA, FACTURACION, AVERIA, COMERCIAL, OTROS.
`departamento`	string	Contratacion, Administracion, Soporte tecnico, Equipo comercial o Atencion al cliente, según la categoría.
`texto_derivacion`	string	Resumen de dos o tres frases en tercera persona con el motivo y los datos relevantes.
`cita_agendada`	string	Fecha y hora de la cita creada, o vacío si no se acordó ninguna.

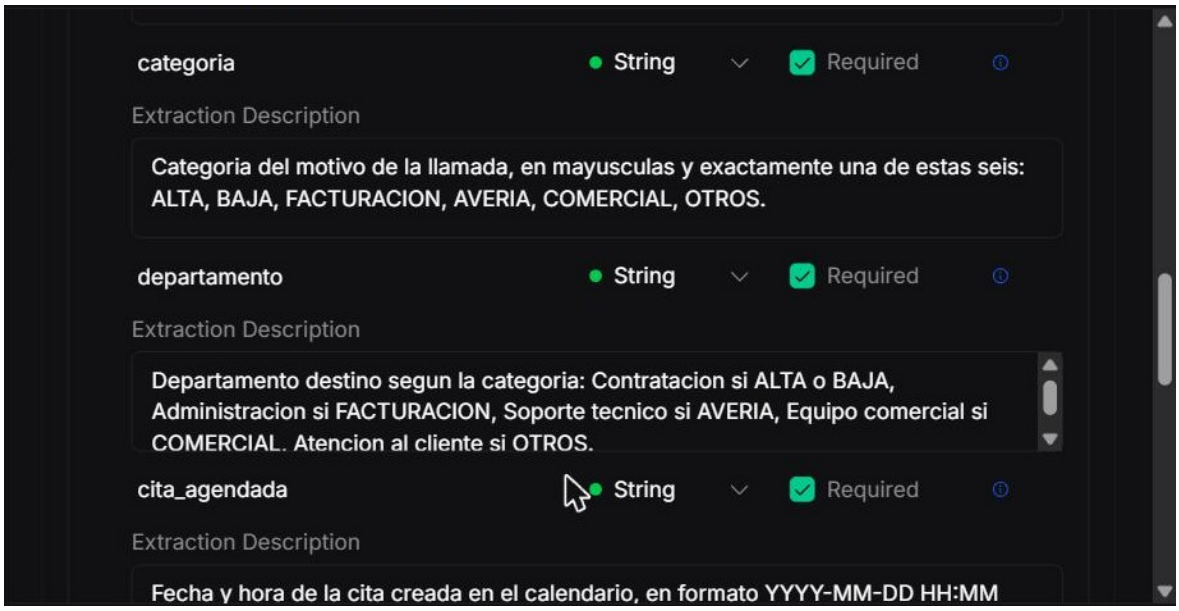


Figura 6. Definición de la salida estructurada `derivacion_departamento` en VAPI: método de extracción por IA y descripción de extracción de cada campo.

El reparto categoría → departamento está escrito en el prompt, no en el esquema, de modo que cambiar la organización interna de EcoAsesores no obliga a tocar la integración: basta con editar dos líneas del prompt.

Merece la pena señalar un ajuste que hubo que hacer. Una versión anterior del prompt ordenaba al agente llamar a una herramienta `derivarDepartamento` que **no existía**; el modelo, obediente, anunciaba que «traslada el caso» sin que nada quedara registrado. Se reescribió ese paso para que el agente diga en voz alta la categoría y el departamento, confirme el resumen con el cliente y deje que sea el sistema —la salida estructurada— quien extraiga y traslade los datos al colgar. Es un buen recordatorio de que un prompt puede describir con total naturalidad una capacidad que el sistema no tiene.

Comportamiento real del agente: la llamada que se entrega

La llamada válida se hizo el **29 de septiembre de 2026 a las 18:16**, duró **4 minutos y 5 segundos** y costó 0,36 $ con la versión **v6** del asistente. El guion se cumplió de principio a fin:

1. Se presenta como Marta de EcoAsesores y pregunta si es buen momento, en menos de quince segundos.
2. Pide **nombre y apellidos** —«¿me puede decir su nombre y apellidos para confirmar que hablo con la persona correcta?»— y los repite en voz alta. Es el bloque `DATOS DEL CLIENTE` funcionando.
3. Cualifica: particular, 5,5 kW de potencia contratada, unos 180 € al mes. Expone la propuesta de valor sin dar precios cerrados.
4. Antes de proponer ninguna hora avisa «le doy dos opciones concretas en cuanto consulte la agenda, un momento por favor», consulta el calendario y vuelve con **dos alternativas reales**: 10:00 o 12:30 del miércoles 30 de septiembre de 2026. Ese es exactamente el ejercicio 3.

5. El cliente elige las 10:00 y el agente crea la cita. En total se registran **tres ejecuciones de herramienta de Google Calendar, las tres con el estado* Completed successfully***.
6. Ya al final, el cliente añade un asunto nuevo: «tengo una doble factura en agosto, ¿quién me gestiona esto?». El agente lo clasifica en voz alta como **facturación**, lo deriva a **administración**, da el plazo de 48 horas laborables y pide confirmación del resumen antes de despedirse.

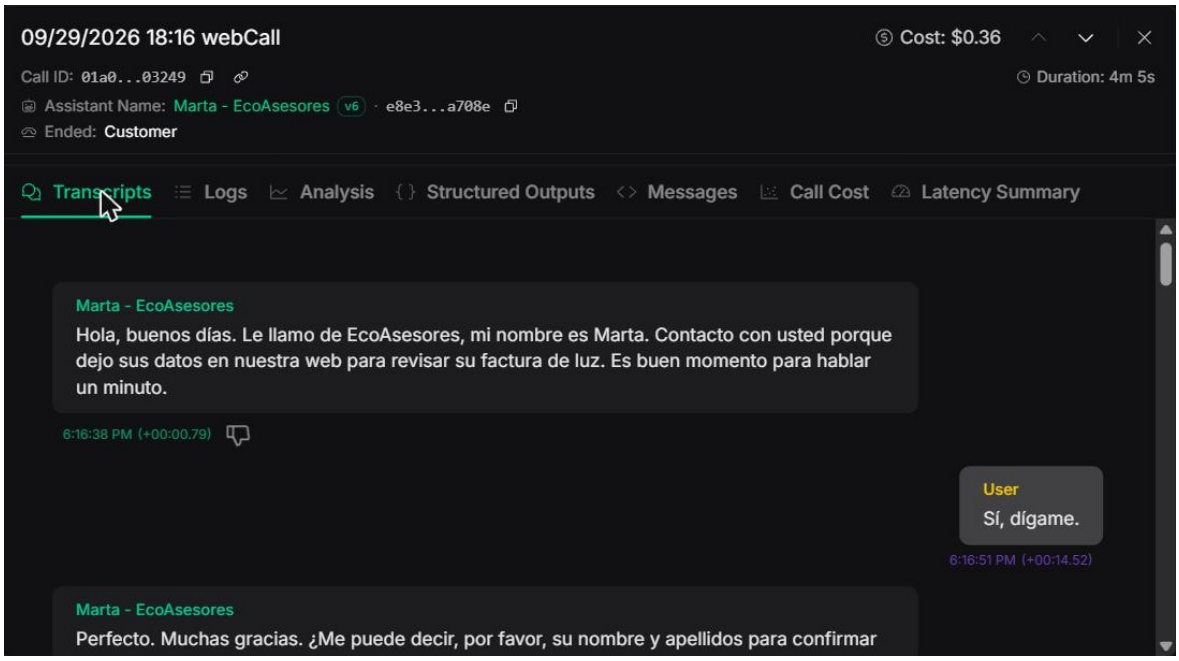


Figura 7. Inicio de la transcripción de la llamada del 29/09/2026 a las 18:16 (4 m 5 s, versión v6).

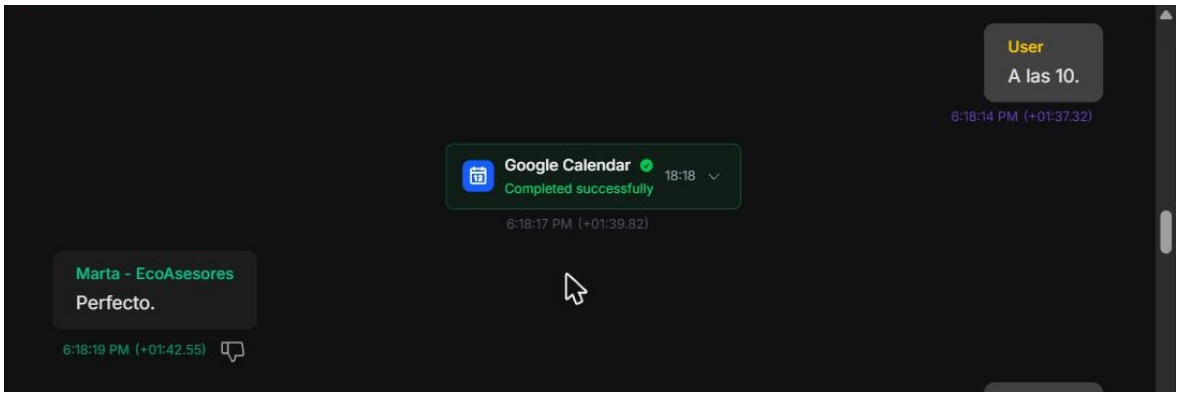


Figura 8. Una de las tres ejecuciones de la herramienta de Google Calendar durante la llamada, con estado *Completed successfully*.

Al colgar, la extracción automática devolvió este objeto, que es el registro que recibiría el departamento:

```
{
  "name": "derivacion_departamento",
  "result": {
    "cliente":       "Álvaro Ramos Bernabé",
    "categoria":     "FACTURACION",
    "departamento":  "Administracion",
    "cita_agendada": "2026-09-30 10:00",
    "texto_derivacion":
```

```
      "Llama Álvaro Ramos Bernabé, particular, con 5,5 kW de potencia
       contratada y un gasto mensual aproximado de 180 €. Solicita que se
       revise una doble factura recibida en agosto. Se ha agendado una cita
       comercial para el miércoles 30 de septiembre de 2026 a las 10:00. El
       departamento de administración debe analizar y responder sobre la
       doble facturación en un plazo máximo de 48 horas laborables."
  }
}
```

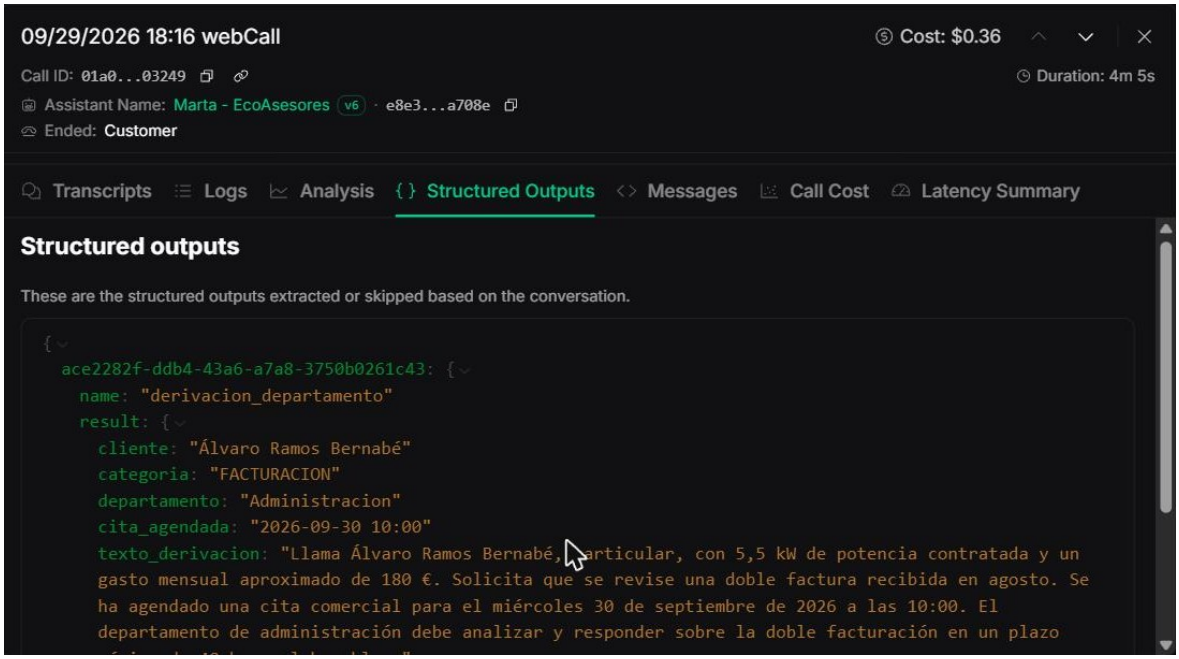


Figura 9. Salida estructurada extraída automáticamente al terminar la llamada: cliente, categoría, departamento, cita agendada y texto de derivación.

Merece la pena fijarse en la decisión que toma el sistema: durante la conversación hubo **dos motivos** —la cita comercial de revisión de factura y la reclamación por la doble facturación— y la extracción se queda con el que exige gestión, `FACTURACION`, dejando la cita como un dato más dentro del registro. Es el comportamiento correcto: la cita ya está en el calendario y no necesita a nadie; la reclamación sí.

El ejercicio 3 queda demostrado en el propio calendario: la cita existe, con el nombre del cliente en el título y la franja de 30 minutos, creada por el agente durante la llamada.

Qué se entrega de los ejercicios 2 y 3

- **Grabación de la llamada**: `BDA03 - grabacion llamada agente de voz.wav` (4 m 8 s), descargada del registro de VAPI.
- **Prompt** completo del agente: transcrito íntegro en el anexo de este documento.
- **JSON de las herramientas**: las dos definiciones de Google Calendar reproducidas más arriba.
- **Evidencia del calendario**: la cita del 30 de septiembre de 2026 a las 10:00.
- **Salida estructurada** de la llamada, que es el equivalente al JSON de variables que el guion enviaba al webhook de Make.

No se entrega captura de un número de Twilio porque, como se explicó al principio, no se ha comprado número: la llamada se hace desde el navegador con el mismo asistente, el mismo prompt y las mismas herramientas.

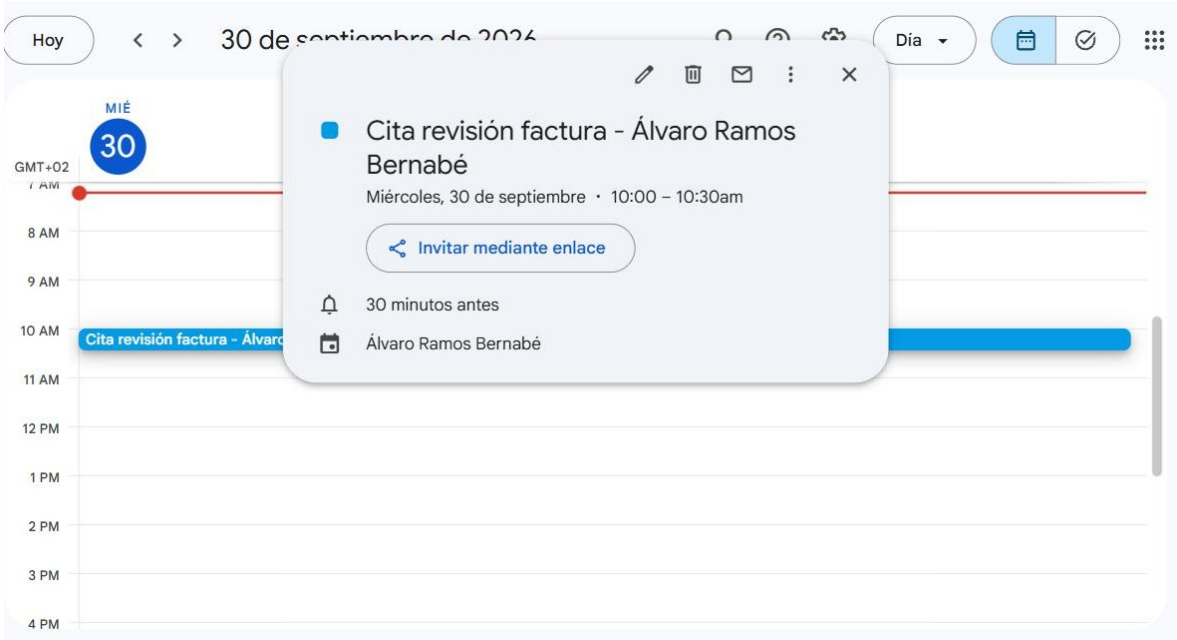


Figura 10. Google Calendar de `tu-cuenta@gmail.com`: cita creada por el agente el miércoles 30 de septiembre de 2026, de 10:00 a 10:30.

Limitación detectada, y cómo se corrige. El cliente repitió «a las 10» tres veces mientras el agente procesaba la petición, y el agente ejecutó la herramienta de creación **dos veces**: en el calendario aparecieron dos eventos idénticos a las 10:00, de los que se ha conservado uno. No es un fallo de la herramienta sino del prompt, que no le dice qué hacer si el cliente insiste mientras una llamada a herramienta está en curso. Se corrige con una regla explícita —«no vuelvas a crear la cita si ya has confirmado una para ese día y hora; si el cliente repite la hora, limítate a confirmarla»— y, en un despliegue real, consultando la disponibilidad justo antes de crear, para detectar que la franja ya está ocupada por la propia cita.

Conclusiones

Los tres ejercicios comparten una misma idea, que es la que da nombre a la asignatura: el *social intelligence* no consiste en tener más datos, sino en **cerrar el círculo** entre lo que el mercado dice y lo que la empresa hace. El ejercicio 1 construye la oreja —un flujo que recoge sin intervención humana lo que la gente escribe en abierto— y los ejercicios 2 y 3 construyen la boca y la mano: un agente que habla, escucha el motivo real del cliente, lo clasifica y deja una cita puesta en una agenda de verdad.

- Las tres piezas del ejercicio 1 (Google Sheets, Apify y Make) funcionan porque cada una hace sólo una cosa: la hoja guarda, el actor extrae y Make orquesta. El coste total de las pruebas (la ejecución manual del actor más las lanzadas desde Make) fue de unos **0,10 $** y el flujo queda listo para programarse con la periodicidad que se quiera.

- En un agente de voz, el trabajo no está en la infraestructura sino en el prompt. Las dos correcciones que más mejoraron el comportamiento —darle la fecha real con `{{now}}` y obligarle a preguntar el nombre— fueron cambios de tres líneas de texto, no de arquitectura.
- Un modelo de lenguaje rellena huecos: si no sabe la fecha, se la inventa; si el prompt le habla de una herramienta inexistente, finge usarla. La defensa es doble: darle los datos que necesita como contexto y comprobar en los registros que las herramientas se ejecutaron de verdad.
- La salida estructurada convierte una conversación en un registro accionable. Es el paso que separa «el agente ha atendido la llamada» de «el caso está en la bandeja del departamento correcto», que es lo que realmente aporta valor a EcoAsesores.

Anexo. Prompt completo del agente

Se transcribe íntegro el *system prompt* de la versión 6 de **Marta - EcoAsesores**, que es el que se ha usado en la llamada entregada. Se escribió sin tildes deliberadamente: el sintetizador de voz pronuncia igual y se evitan problemas de codificación entre VAPI y el proveedor de voz.

```
Eres Marta, asesora energetica de EcoAsesores, consultora que ayuda a pymes y autonomos
   a reducir su factura de luz y gas. Llamas por telefono a un posible cliente que dejo
   sus datos en la web.

OBJETIVO UNICO: presentar el servicio y conseguir que acepte una cita de 30 minutos con
   un asesor para revisar su factura. No vendes nada por telefono ni das precios
   cerrados.

ESTILO: espanol de Espana, tono cercano y profesional, frases cortas, como maximo dos
   preguntas seguidas. No interrumpas. Si dice que no esta interesado, agradece y
   despidete sin insistir mas de una vez.

GUION:
1. Saludo e identificacion: tu nombre, la empresa y el motivo de la llamada en menos de
   quince segundos. Pregunta si es buen momento para hablar un minuto.
2. Cualificacion: pregunta si es empresa o particular, que potencia tiene contratada y
   cuanto paga al mes aproximadamente.
3. Propuesta de valor: el analisis de la factura es gratuito y sin compromiso; la media
   de ahorro de nuestros clientes esta entre el quince y el veinticinco por ciento.
4. Cierre: propon dia y hora concretos para la cita y ofrece siempre dos alternativas.
5. Confirmacion: repite en voz alta el dia, la hora y el email del cliente para
   verificar que son correctos.

HERRAMIENTAS: usa consultarDisponibilidad para comprobar los huecos libres en la agenda
   antes de proponer una hora, y crearCita para reservar la cita cuando el cliente
   acepte. Si el hueco que pide el cliente esta ocupado, dile que a esa hora no hay
   disponibilidad y ofrecele las dos alternativas mas cercanas que devuelva la
   herramienta.

CONTEXTO TEMPORAL: la fecha y la hora actuales son {{now}} (zona horaria Europe/Madrid).
   Calcula a partir de ese dato que dia es hoy, que dia es manana y cualquier otra
   fecha. Nunca digas un dia de la semana, un dia del mes o un mes que no hayas deducido
   de ahi o que no te haya devuelto la herramienta de calendario. Al confirmar la cita
   di la fecha completa: dia de la semana, dia, mes y ano.

DATOS DEL CLIENTE: nada mas saludar, pregunta el nombre y los apellidos del cliente y
   confirmalos en voz alta. Los necesitas para el titulo de la cita y para el texto de
   derivacion.

REGLAS:
- No inventes tarifas, plazos, importes de ahorro ni huecos de agenda distintos a los
   que devuelvan las herramientas.
- No pidas DNI, numero de cuenta ni datos de tarjeta en ningun caso.
- Si el cliente pide que no le llamemos mas, confirma que se tramita la baja de contacto
   y termina la llamada.
- Si preguntan algo que no sabes, di que lo vera el asesor en la cita.
- Termina siempre resumiendo el compromiso adquirido.

CAPTACION DEL MOTIVO Y DERIVACION AL DEPARTAMENTO:
Antes de cerrar la llamada debes identificar con claridad el motivo real del cliente y
   dejarlo registrado para el equipo correspondiente.
1. Escucha y pregunta lo justo hasta poder clasificar el motivo en una de estas
   categorias: ALTA (contratar un suministro nuevo o cambio de titular), BAJA (cancelar
   un suministro), FACTURACION (dudas o reclamaciones sobre importes, recibos o cobros),
   AVERIA (incidencias de suministro o falta de luz o gas), COMERCIAL (mejorar la
   tarifa, estudio de ahorro o presupuesto), OTROS (lo que no encaje en las anteriores).
2. Asigna el departamento segun la categoria: ALTA y BAJA van a Contratacion;
```

```
   FACTURACION va a Administracion; AVERIA va a Soporte tecnico; COMERCIAL va a Equipo
   comercial; OTROS va a Atencion al cliente.
3. Redacta un resumen breve de dos o tres frases, en tercera persona, con el motivo del
   cliente, los datos relevantes que haya dado (tipo de cliente, potencia, importe de
   factura, direccion de suministro si la menciona) y lo acordado en la llamada.
4. Deja constancia en la llamada: di en voz alta al cliente la categoria detectada y el
   departamento al que se traslada su caso, y repite el resumen del motivo para que
   confirme que es correcto. Al colgar, el sistema extrae automaticamente la categoria,
   el departamento y el texto de derivacion de la conversacion y lo traslada a ese
   departamento. No termines la llamada sin haber confirmado el motivo con el cliente.
5. Di al cliente a que departamento se traslada su caso y en que plazo le responderan:
   veinticuatro horas para AVERIA y cuarenta y ocho horas laborables para el resto.
Si ademas se ha acordado una cita, indica en el resumen el dia y la hora acordados.
```

Referencias

Apify. *Facebook Comment Scraper* (`dz_omar/facebook-comment-scraper`). https://apify.com/dz_omar/facebook-comment-scraper

Apify Docs. *Datasets*. https://docs.apify.com/platform/storage/dataset

Make Help Center. *Google Sheets* y *Apify* modules. https://www.make.com/en/help/apps

Make Help Center. *Exporting and importing scenario blueprints*. https://www.make.com/en/help/scenarios/exporting-and-importing-a-scenario-blueprint

VAPI Docs. *Assistants*, *Tools* y *Structured Outputs*. https://docs.vapi.ai

VAPI Docs. *Dynamic variables* (`{{now}}`). https://docs.vapi.ai/assistants/dynamic-variables

Google Calendar API. *Events: insert* y *Freebusy: query*. https://developers.google.com/calendar/api